

UNIVERSIDADE FEDERAL DE LAVRAS

Programação Aplicada à Engenharia

Projeto Final — Documentação para o perfil de desenvolvedor

CANTEIRO

Jogo de encaixe de blocos com restrição de estabilidade estrutural

INTEGRANTES DO GRUPO

Nome completo	Matrícula

Lavras, Minas Gerais
2026

Sumário

1. Introdução.....	1
1.1. Contextualização.....	1
1.2. Motivação e justificativa.....	1
1.2.1. Trabalhos correlatos.....	1
1.3. Objetivos.....	2
1.3.1. Objetivo geral.....	2
1.3.2. Objetivos específicos.....	2
1.4. Escopo e público-alvo.....	2
1.5. Organização do documento.....	2
2. Levantamento de requisitos.....	3
2.1. Metodologia adotada.....	3
2.2. Visão geral do produto.....	3
2.2.1. Atores do sistema.....	3
2.3. Regras de negócio.....	3
2.4. Requisitos funcionais.....	4
2.5. Requisitos não funcionais.....	5
2.6. Diagrama de casos de uso.....	6
2.6.1. Detalhamento do caso de uso «Movimentar e rotacionar peça».....	7
2.6.2. Detalhamento do caso de uso «Sofrer colapso estrutural».....	7
2.7. Modelo de classes.....	7
2.7.1. Hierarquias de peças e materiais.....	7
2.7.2. Núcleo do jogo e estruturas de dados.....	9
2.7.3. Estruturas de dados empregadas.....	9
2.8. Diagrama de estados da partida.....	10
2.9. Organização em pacotes.....	11
2.10. Dados persistidos.....	12
2.11. Premissas e restrições.....	12
2.12. Rastreabilidade entre requisitos e conteúdo da disciplina.....	12
3. Estratégias de codificação.....	13
3.1. Plataforma e ferramentas.....	13
3.2. Arquitetura adotada.....	13
3.3. Convenções de código e documentação.....	14
3.4. Uso de herança e polimorfismo.....	14
3.5. Algoritmos centrais.....	15
3.5.1. Ciclo de atualização.....	15
3.5.2. Detecção de colisão.....	16
3.5.3. Rotação com deslocamento corretivo.....	16
3.5.4. Cálculo incremental do centro de massa.....	16
3.5.5. Colapso estrutural.....	17
3.6. Tratamento de exceções.....	17
3.7. Estratégia de testes.....	17
3.8. Controle de versão e trabalho em equipe.....	17

4. Resultados esperados.....	17
4.1. Produto final previsto.....	17
4.2. Interface prevista.....	18
4.2.1. Telas previstas.....	18
4.3. Comportamento esperado da mecânica de estabilidade.....	19
4.4. Progressão e balanceamento previstos.....	19
4.5. Relatório de fim de partida.....	20
4.6. Critérios de aceitação.....	20
4.7. Métricas de desempenho esperadas.....	21
4.8. Plano de verificação e testes.....	21
4.9. Validação com jogadores.....	22
4.10. Entregáveis previstos.....	22
4.11. Cronograma previsto.....	23
4.12. Riscos identificados.....	23
4.13. Limitações reconhecidas.....	23
5. Conclusão.....	24

1. Introdução

1.1. Contextualização

Jogos digitais são um dos domínios mais férteis para o ensino de programação orientada a objetos. Um jogo reúne, em um único artefato de software, entidades com comportamento próprio, um laço de simulação que evolui no tempo, uma interface gráfica que precisa responder ao usuário sem travar e um conjunto de regras que muda conforme o estado do sistema. Essa combinação exige do desenvolvedor exatamente aquilo que a disciplina procura exercitar: modelagem por classes, uso de herança para fatorar comportamento comum, polimorfismo para tratar objetos diferentes de maneira uniforme e escolha consciente de estruturas de dados para cada necessidade de armazenamento e acesso.

Os jogos de encaixe de blocos, popularizados na década de 1980, formam um gênero em que peças geométricas descem por um tabuleiro retangular e devem ser posicionadas pelo jogador de modo a formar linhas horizontais completas, que são eliminadas e liberam espaço. A mecânica é simples de enunciar e difícil de dominar, o que explica sua longevidade. Do ponto de vista computacional, porém, o gênero na sua forma tradicional é um problema já bastante explorado e cujas soluções estão amplamente documentadas.

O projeto descrito neste documento parte desse gênero, mas o submete a uma restrição adicional de natureza física, tomada de empréstimo da análise estrutural: a pilha de blocos construída pelo jogador possui massa, e essa massa precisa permanecer equilibrada sobre a base de apoio. A adição transforma o problema. Não basta mais preencher lacunas; é preciso decidir onde colocar carga.

1.2. Motivação e justificativa

A motivação do grupo tem três origens. A primeira é acadêmica: a disciplina exige a aplicação de pilhas, filas, listas, tabelas hash, herança e polimorfismo, e um jogo desse tipo oferece um uso natural para cada uma dessas construções, sem que seja necessário forçar sua presença. A fila de próximas peças, a pilha de peças reservadas, a matriz do tabuleiro e o dicionário de materiais existem porque o jogo precisa deles, não porque o enunciado os pede.

A segunda origem é a própria natureza do curso de engenharia. Um jogo em que o jogador precisa observar o centro de massa de uma estrutura, distribuir materiais de densidades diferentes e evitar que a construção tombe conecta o exercício de programação a um conteúdo que os integrantes já estudam em mecânica dos sólidos e estática. O sistema deixa de ser um exercício genérico e passa a ser uma pequena ferramenta de intuição sobre equilíbrio.

A terceira é a exigência de originalidade. Implementações do gênero clássico existem aos milhares em repositórios públicos e em livros didáticos, o que torna qualquer versão fiel ao original difícil de defender como trabalho autoral. A mecânica de estabilidade estrutural proposta aqui, com materiais de densidade distinta, cálculo contínuo do centro de massa e colapso parcial da pilha, não foi localizada pelo grupo em nenhuma implementação existente do gênero e constitui o diferencial do projeto.

1.2.1. Trabalhos correlatos

O grupo levantou três categorias de trabalhos próximos. A primeira reúne as implementações fiéis do gênero clássico, abundantes em repositórios públicos e em capítulos de livros de introdução a jogos: são referências úteis para a mecânica de encaixe, rotação e eliminação de linhas, mas não trazem qualquer elemento físico e, por serem tão numerosas, não poderiam ser reproduzidas sem comprometer a originalidade exigida.

A segunda categoria reúne jogos comerciais de empilhamento com física, em que a simulação é contínua e baseada em corpos rígidos, com colisão, atrito e rotação livres. O CANTEIRO se afasta desse modelo: mantém a grade discreta do gênero de encaixe e aplica apenas um critério agregado de equilíbrio, o que preserva a precisão do posicionamento e mantém a complexidade compatível com o prazo da disciplina.

A terceira são as ferramentas didáticas de estática que exibem centro de gravidade e polígono de apoio: instrutivas, mas sem pressão de tempo, pontuação ou decisão sob incerteza. A proposta ocupa o espaço vazio entre as três — a precisão da grade discreta do primeiro grupo, a consequência física do segundo e o conteúdo conceitual do terceiro.

1.3. Objetivos

1.3.1. Objetivo geral

Desenvolver, em linguagem Java, um jogo de encaixe de blocos com interface gráfica, denominado CANTEIRO, no qual a validade da estrutura construída pelo jogador depende do equilíbrio de sua distribuição de massa, aplicando de forma justificada os conceitos de programação orientada a objetos e de estruturas de dados estudados na disciplina.

1.3.2. Objetivos específicos

- Modelar as sete formas de peça e os quatro materiais de construção por meio de duas hierarquias de classes independentes, exercitando herança e polimorfismo.
- Implementar um motor de jogo capaz de tratar queda, movimentação, rotação com correção de posição, detecção de colisão e eliminação de linhas em tempo real.
- Implementar o cálculo incremental do centro de massa da estrutura e a regra de colapso associada.
- Empregar fila para a sequência de peças futuras, pilha para a peça reservada e para o histórico de jogadas, lista para a composição das peças e tabela hash para o catálogo de materiais e para o ranking.
- Construir uma interface gráfica em Swing com laço de atualização estável e realimentação visual do índice de estabilidade.
- Persistir ranking e repetições de partida em arquivo, permitindo reexecutar uma partida encerrada.
- Documentar integralmente as classes públicas com Javadoc e cobrir as regras críticas com testes automatizados.

1.4. Escopo e público-alvo

O sistema é um aplicativo de mesa, monousuário e executado localmente, sem qualquer dependência de rede ou de servidor. Destina-se a jogadores casuais a partir de doze anos e, em segundo plano, a estudantes de engenharia que queiram uma noção intuitiva de equilíbrio de cargas. Estão fora do escopo desta versão: modo multijogador, adversário controlado por computador, edição de fases pelo usuário e versão para dispositivos móveis.

1.5. Organização do documento

A seção 2 apresenta o levantamento de requisitos, com as regras de negócio, os requisitos funcionais e não funcionais e os diagramas de casos de uso, de classes, de estados e de pacotes. A seção 3 descreve as estratégias de codificação adotadas, incluindo arquitetura, convenções, algoritmos centrais e plano de testes. A seção 4 detalha os resultados esperados, os critérios de aceitação e o cronograma. A seção 5 encerra o documento com as considerações finais.

2. Levantamento de requisitos

2.1. Metodologia adotada

Como o produto não possui um cliente externo, o levantamento foi conduzido pelo próprio grupo em três rodadas de trabalho. Na primeira, foi feita uma sessão de análise do gênero: os integrantes jogaram implementações livres de jogos de encaixe e registraram, em lista, todo comportamento observado que consideraram essencial à experiência. Na segunda rodada, essa lista foi confrontada com a mecânica de estabilidade proposta, e cada item foi classificado como mantido, alterado ou descartado. Na terceira, os itens sobreviventes foram reescritos no formato de requisitos verificáveis e priorizados pela técnica MoSCoW, que separa o que é obrigatório do que é desejável.

2.2. Visão geral do produto

CANTEIRO simula a construção de uma edificação. O jogador recebe, um a um, elementos construtivos formados por quatro blocos unitários e deve encaixá-los na estrutura em erguimento. Cada elemento é fabricado em um material — madeira, alvenaria, concreto ou aço — que define sua densidade, sua cor e sua pontuação. Linhas horizontais completas representam pavimentos concluídos e são eliminadas, fazendo a estrutura acima descer.

Paralelamente, o sistema mantém a análise estrutural da pilha. A cada elemento fixado, o centro de massa é recalculado e comparado ao eixo da base de apoio. O afastamento entre os dois é convertido em um índice de estabilidade exibido continuamente. Quando o afastamento ultrapassa o limite tolerado pelo nível, ocorre o colapso: as camadas acima da linha crítica desabam, os blocos são reacomodados por gravidade e o jogador perde pontos. A partida termina quando um novo elemento não encontra espaço para ser gerado ou quando um colapso ocorre com a estrutura acima da altura crítica.

2.2.1. Atores do sistema

Ator	Descrição
Jogador	Único ator humano. Inicia partidas, controla os elementos em queda, consulta o ranking e reproduz partidas anteriores.
Relógio do jogo	Ator temporal. Dispara a atualização do estado em intervalos regulares, provocando a queda automática dos elementos independentemente de ação do jogador.
Sistema de arquivos	Ator externo. Fornece e recebe os dados persistidos: catálogo de materiais, ranking de pontuações e arquivos de repetição.

2.3. Regras de negócio

As regras abaixo condicionam o comportamento do sistema e são referenciadas pelos requisitos funcionais.

Código	Regra
RN01	O tabuleiro possui 10 colunas e 20 linhas visíveis, mais 2 linhas ocultas de geração no topo.
RN02	Todo elemento é composto por exatamente quatro blocos unitários conexos, em uma das sete formas canônicas (I, O, T, S, Z, J, L).
RN03	O material do elemento é sorteado entre os materiais liberados na fase corrente e determina sua densidade, cor e multiplicador de pontos.
RN04	A sequência de elementos futuros é gerada pelo método da sacola: os sete formatos são embaralhados e distribuídos antes que qualquer formato se repita, garantindo distribuição uniforme.

Código	Regra
RN05	O jogador pode reservar um elemento por vez. A troca só pode ser feita uma vez a cada elemento gerado.
RN06	Uma linha é eliminada quando todas as suas 10 posições estão ocupadas. Eliminações simultâneas de 2, 3 ou 4 linhas recebem bônus progressivo.
RN07	A massa de um bloco é o produto da densidade do seu material pelo volume unitário. A massa da estrutura é a soma das massas de seus blocos.
RN08	O centro de massa horizontal é a média das posições das colunas dos blocos, ponderada pela massa de cada bloco.
RN09	O desvio é o módulo da diferença entre o centro de massa e o eixo central da base de apoio, medido em colunas.
RN10	O índice de estabilidade é 100 % quando o desvio é nulo e 0 % quando o desvio atinge o limite do nível, variando linearmente entre esses extremos.
RN11	Ocorre colapso quando o desvio ultrapassa o limite do nível. No colapso, os blocos acima da linha de maior concentração de massa são desprendidos e reacomodados por gravidade coluna a coluna.
RN12	Cada colapso subtrai da pontuação o equivalente a duas linhas eliminadas e incrementa o contador de colapsos da partida.
RN13	A partida termina se um elemento recém-gerado colidir imediatamente com a estrutura existente, ou se um colapso ocorrer com a pilha ocupando mais de 18 linhas.
RN14	O nível avança a cada 10 linhas eliminadas. A cada nível, o intervalo de queda diminui e o limite de desvio tolerado é reduzido.
RN15	Somente pontuações de partidas concluídas sem uso do recurso de desfazer são registradas no ranking.

2.4. Requisitos funcionais

Os requisitos abaixo estão classificados por prioridade: E indica requisito essencial, presente na versão mínima entregável; D indica requisito desejável, implementado se houver disponibilidade de prazo.

Código	Descrição	Prior.	Regras
RF01	O sistema deve exibir um menu principal com as opções de nova partida, ranking, repetições, configurações e sair.	E	—
RF02	O sistema deve permitir ao jogador escolher entre três dificuldades, que definem velocidade inicial de queda e limite de desvio.	E	RN14
RF03	O sistema deve gerar elementos de forma contínua enquanto a partida estiver em andamento.	E	RN02, RN04
RF04	O sistema deve atribuir a cada elemento gerado um material sorteado entre os liberados na fase.	E	RN03
RF05	O sistema deve exibir ao jogador os três próximos elementos da fila, com sua forma e seu material.	E	RN04
RF06	O sistema deve deslocar o elemento em queda uma linha para baixo a cada intervalo definido pelo nível.	E	RN14
RF07	O sistema deve permitir mover o elemento em queda para a esquerda e para a direita, respeitando os limites do tabuleiro e os blocos já fixados.	E	RN01
RF08	O sistema deve permitir girar o elemento em queda nos dois sentidos, aplicando	E	RN02

Código	Descrição	Prior.	Regras
	deslocamento corretivo quando a rotação simples resultar em sobreposição.		
RF09	O sistema deve permitir a queda instantânea do elemento até a primeira posição de apoio.	E	—
RF10	O sistema deve permitir reservar o elemento em queda e trocá-lo pelo elemento previamente reservado, uma vez por elemento.	E	RN05
RF11	O sistema deve fixar o elemento no tabuleiro quando este colidir com o fundo ou com um bloco já fixado.	E	—
RF12	O sistema deve identificar e eliminar todas as linhas completas após a fixação de um elemento, deslocando para baixo as linhas superiores.	E	RN06
RF13	O sistema deve calcular a pontuação considerando o número de linhas eliminadas simultaneamente, o material predominante e o nível corrente.	E	RN06, RN03
RF14	O sistema deve recalculer o centro de massa da estrutura sempre que a composição do tabuleiro for alterada.	E	RN07, RN08
RF15	O sistema deve exibir continuamente o índice de estabilidade, o desvio corrente e o limite tolerado.	E	RN09, RN10
RF16	O sistema deve sinalizar visualmente ao jogador a aproximação do limite de desvio antes que o colapso ocorra.	E	RN10
RF17	O sistema deve executar o colapso da estrutura quando o desvio ultrapassar o limite, reacomodando os blocos desprendidos.	E	RN11, RN12
RF18	O sistema deve avançar de nível a cada dez linhas eliminadas, ajustando velocidade e limite de desvio.	E	RN14
RF19	O sistema deve permitir pausar e retomar a partida.	E	—
RF20	O sistema deve encerrar a partida nas condições de fim de jogo e exibir a tela de resultado.	E	RN13
RF21	O sistema deve registrar a pontuação obtida no ranking persistente, associada ao nome informado pelo jogador.	E	RN15
RF22	O sistema deve exibir o ranking com as dez melhores pontuações registradas.	E	RN15
RF23	O sistema deve registrar em pilha todas as jogadas executadas durante a partida.	E	—
RF24	O sistema deve permitir reproduzir passo a passo uma partida encerrada, a partir do histórico registrado.	D	RF23
RF25	O sistema deve exibir, ao final da partida, um relatório com a evolução do índice de estabilidade e a distribuição de materiais utilizados.	D	—
RF26	O sistema deve oferecer, no modo treino, o desfazer da última jogada.	D	RF23, RN15
RF27	O sistema deve permitir configurar as teclas de comando.	D	—
RF28	O sistema deve oferecer efeitos sonoros para fixação, eliminação de linha e colapso.	D	—

2.5. Requisitos não funcionais

Código	Categoria	Descrição
RNF01	Desempenho	O sistema deve manter taxa de atualização de 60 quadros por segundo em máquina com processador de dois núcleos e 4 GB de memória.

Código	Categoria	Descrição
RNF02	Desempenho	O recálculo do centro de massa deve ser incremental, executado em tempo constante por bloco alterado, e não deve percorrer todo o tabuleiro a cada quadro.
RNF03	Desempenho	O tempo entre o pressionamento de uma tecla e a resposta visual não deve exceder 50 ms.
RNF04	Portabilidade	O sistema deve executar sem alteração de código em Windows, Linux e macOS, exigindo apenas Java 17 ou superior.
RNF05	Usabilidade	Os comandos devem ser aprendidos sem consulta a manual, com legenda visível na própria tela de jogo.
RNF06	Usabilidade	As cores dos materiais devem ser distinguíveis também por padrão de textura, atendendo jogadores com daltonismo.
RNF07	Manutenibilidade	Todas as classes e métodos públicos devem possuir documentação Javadoc completa, com descrição de parâmetros, retorno e exceções.
RNF08	Manutenibilidade	As regras do jogo devem residir integralmente na camada de modelo, sem qualquer dependência de classes gráficas, permitindo execução dos testes sem interface.
RNF09	Manutenibilidade	Nenhum método deve exceder 40 linhas úteis e nenhuma classe deve exceder 400 linhas.
RNF10	Confiabilidade	As regras de colisão, rotação, eliminação de linhas e cálculo do centro de massa devem possuir cobertura por testes automatizados.
RNF11	Confiabilidade	Falha na leitura dos arquivos de ranking ou de configuração não deve impedir a execução do jogo; o sistema deve recorrer a valores padrão.
RNF12	Segurança	Arquivos de dados devem ser validados na leitura; conteúdo malformatado deve ser rejeitado com registro em log, sem interromper a aplicação.

2.6. Diagrama de casos de uso

O diagrama da Figura 1 sintetiza as interações previstas entre o jogador e o sistema. O caso de uso de colapso é uma extensão do acompanhamento da estabilidade, pois ocorre automaticamente quando a condição da regra RN11 é satisfeita, sem ação direta do jogador.

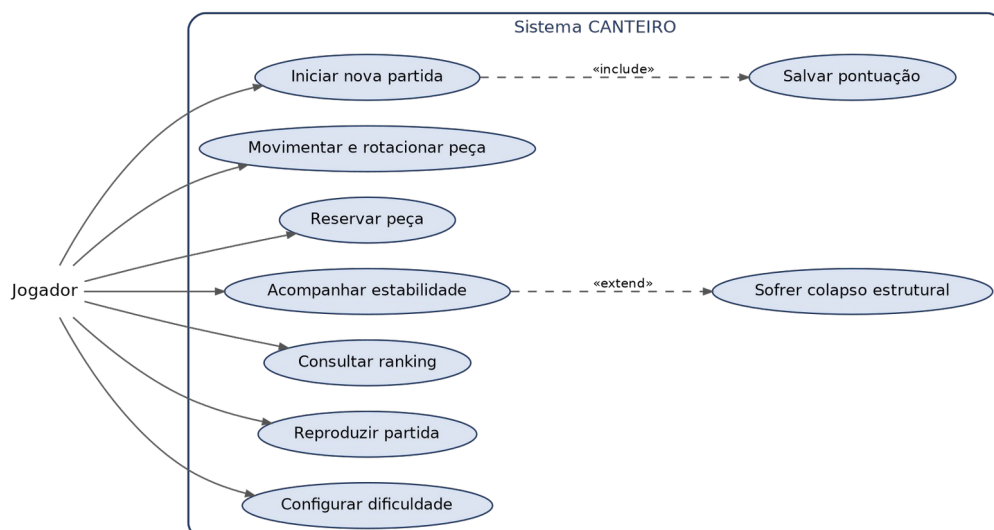


Figura 1 — Diagrama de casos de uso do sistema CANTEIRO.

2.6.1. Detalhamento do caso de uso «Movimentar e rotacionar peça»

Campo	Conteúdo
Ator principal	Jogador
Pré-condição	Existe uma partida em andamento, não pausada, com um elemento em queda.
Fluxo principal	1. O jogador aciona uma tecla de comando. 2. O sistema enfileira o comando para o próximo ciclo de atualização. 3. O sistema calcula a posição resultante do elemento. 4. O sistema verifica colisão com as bordas e com os blocos fixados. 5. Não havendo colisão, o sistema aplica a nova posição. 6. O sistema redesenha o tabuleiro.
Fluxo alternativo A	4a. Há colisão em rotação. O sistema tenta deslocar o elemento uma e duas colunas para cada lado e uma linha para cima; aplicando a primeira posição válida encontrada.
Fluxo alternativo B	4b. Há colisão em movimento lateral. O sistema descarta o comando e mantém a posição anterior, sem informar erro.
Pós-condição	O elemento encontra-se em posição válida do tabuleiro.

2.6.2. Detalhamento do caso de uso «Sofrer colapso estrutural»

Campo	Conteúdo
Ator principal	Sistema (disparado pelo relógio do jogo)
Pré-condição	Um elemento acaba de ser fixado ou linhas acabam de ser eliminadas.
Fluxo principal	1. O sistema recalcula a massa total, o centro de massa e o desvio em relação ao eixo da base. 2. Constatado desvio superior ao limite do nível, o sistema identifica a linha crítica e desprende os blocos acima dela. 3. O sistema reacomoda os blocos desprendidos por gravidade, coluna a coluna. 4. O sistema aplica a penalidade de pontuação, incrementa o contador de colapsos, exibe a animação e retoma a partida.
Fluxo alternativo	3a. A pilha ocupa mais de 18 linhas. O sistema encerra a partida conforme RN13.
Pós-condição	A estrutura encontra-se novamente dentro do limite de desvio, ou a partida foi encerrada.

2.7. Modelo de classes

O modelo de classes foi dividido em duas figuras por clareza. A Figura 2 apresenta as hierarquias que concentram o uso de herança e polimorfismo; a Figura 3 apresenta o núcleo do jogo e as classes responsáveis pelas estruturas de dados.

2.7.1. Hierarquias de peças e materiais

A decisão de modelagem central é a separação entre forma e material. A forma determina como o elemento ocupa o espaço e como gira; o material determina sua massa e sua pontuação. São dimensões independentes: qualquer forma pode ser fabricada em qualquer material. Tratá-las em uma única hierarquia exigiria vinte e oito classes; em duas hierarquias com composição, exige onze.

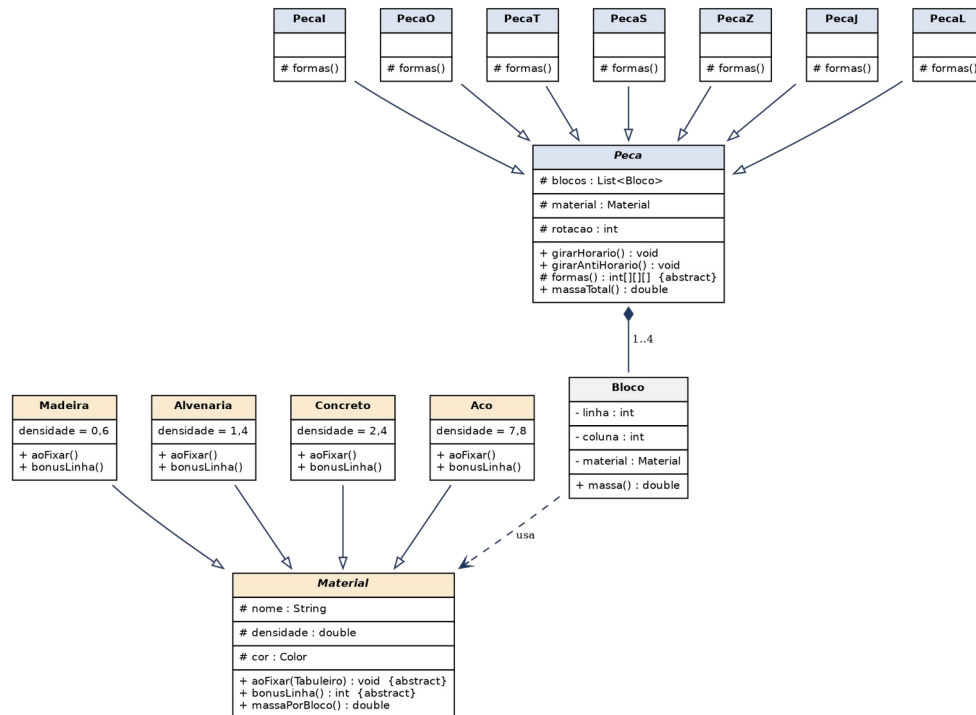


Figura 2 — Hierarquias de peças e de materiais, com a classe **Bloco** fazendo a ligação entre elas.

A classe abstrata **Peca** concentra tudo o que é comum: a lista de blocos, a referência ao material, o índice de rotação corrente e os métodos de giro. O que varia é apenas a matriz de ocupação de cada rotação, fornecida pelo método abstrato `formas()`, implementado por cada subclasse. O motor do jogo nunca precisa saber com qual das sete formas está lidando.

A classe abstrata **Material** segue a mesma lógica. Ela guarda nome, densidade e cor, e declara como abstratos o método `aoFixar()`, que permite a cada material reagir de forma própria no instante em que o elemento é assentado, e o método `bonusLinha()`, que define quanto aquele material acrescenta à pontuação de uma linha eliminada. O aço, por exemplo, oferece bônus alto por ser o material mais difícil de equilibrar.

2.7.2. Núcleo do jogo e estruturas de dados

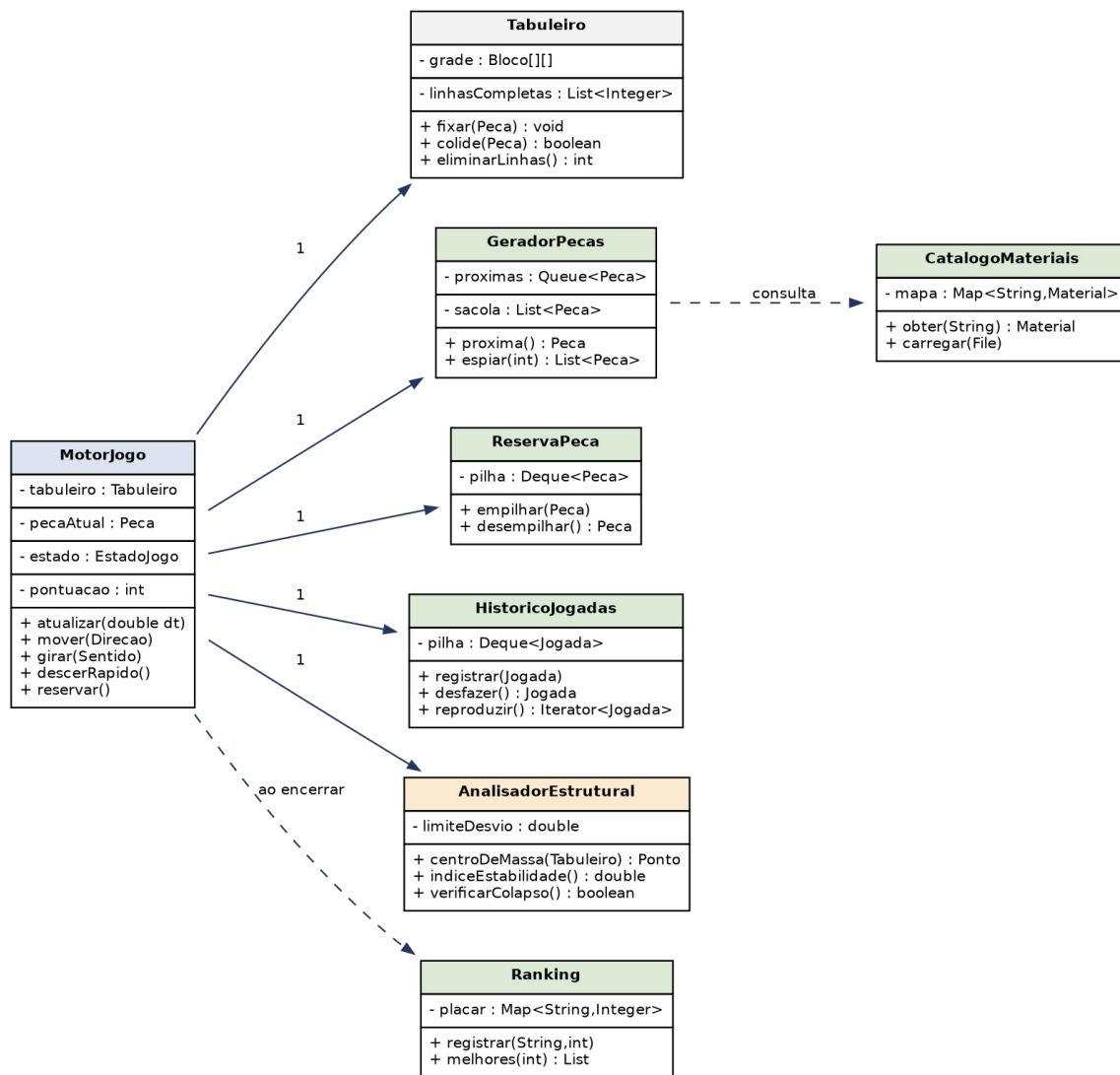


Figura 3 — Classes do núcleo do jogo e sua relação com as estruturas de dados.

A classe **MotorJogo** é o coordenador. Ela não implementa regras de tabuleiro nem cálculos físicos: delega. Recebe comandos do controlador, aplica-os sobre o elemento em queda, consulta o **Tabuleiro** para saber se há colisão, solicita ao **AnalisadorEstrutural** a verificação de equilíbrio e pede ao **GeradorPecas** o próximo elemento. Essa concentração de responsabilidade de coordenação em uma única classe mantém as demais pequenas e testáveis isoladamente.

2.7.3. Estruturas de dados empregadas

Necessidade	Estrutura	Justificativa
Sequência de elementos futuros	Fila (Queue / ArrayDeque)	O consumo é estritamente pela ordem de chegada e a inserção sempre ocorre no fim. A fila expressa exatamente essa disciplina e oferece remoção em tempo constante.
Elemento reservado	Pilha (Deque)	A operação de reserva é do tipo último a entrar, primeiro a sair. Embora a versão mínima guarde um único elemento, a pilha permite estender a regra para reservas múltiplas sem reescrever a classe.

Necessidade	Estrutura	Justificativa
Histórico de jogadas	Pilha (Deque)	O desfazer exige acesso ao topo, e a reprodução exige percorrer o histórico em ordem inversa de empilhamento. Ambas são operações naturais da pilha.
Ocupação do tabuleiro	Matriz de objetos	O acesso por linha e coluna é constante e a verificação de linha completa percorre apenas dez posições. Uma lista encadeada exigiria travessia.
Blocos de um elemento	Lista (ArrayList)	O número de blocos é fixo em quatro, mas o acesso é sempre sequencial durante o desenho e a verificação de colisão.
Catálogo de materiais	Tabela hash (HashMap)	Os materiais são carregados de arquivo e recuperados pelo seu código textual a cada geração de elemento. A busca por chave em tempo constante evita percorrer a lista de materiais milhares de vezes por partida.
Ranking de pontuações	Tabela hash + lista ordenada	A tabela associa nome do jogador à sua melhor pontuação, evitando duplicatas. A exibição ordenada é obtida em uma lista derivada, ordenada apenas quando a tela de ranking é aberta.
Massa acumulada por coluna	Vetor de acumuladores	Permite atualizar o centro de massa em tempo constante por bloco alterado, atendendo ao requisito RNF02.

2.8. Diagrama de estados da partida

O comportamento do jogo é governado por uma máquina de estados finita. Modelar explicitamente esses estados evita a proliferação de variáveis booleanas de controle e torna o laço de atualização legível: a cada quadro, o motor consulta o estado corrente e executa apenas as transições nele previstas.

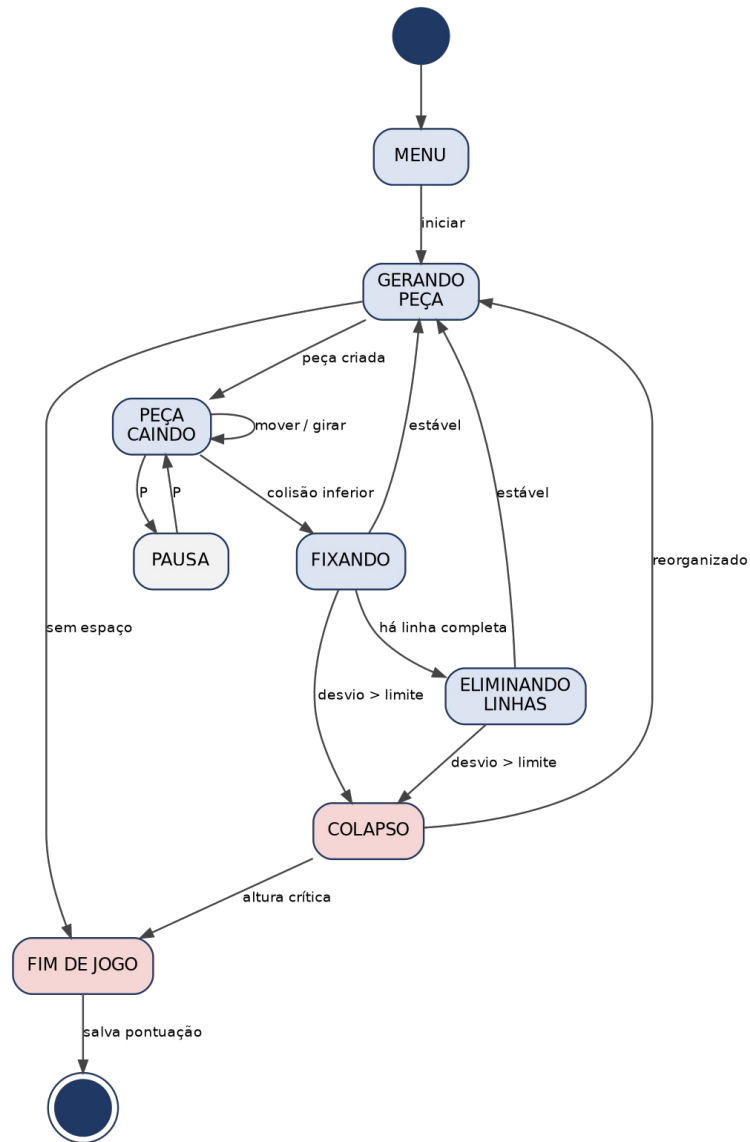


Figura 4 — Máquina de estados da partida.

2.9. Organização em pacotes

A separação em pacotes reflete a arquitetura em camadas e impõe, por construção, a restrição do requisito RNF08: o pacote de modelo não importa nenhuma classe do pacote de visão, o que garante que as regras do jogo possam ser exercitadas por testes sem que uma janela seja aberta.

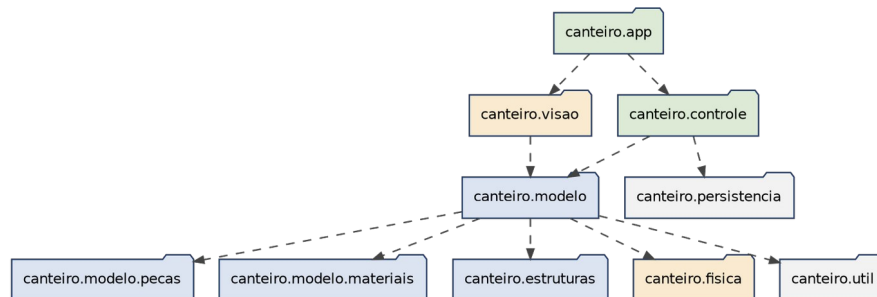


Figura 5 — Diagrama de pacotes e suas dependências.

2.10. Dados persistidos

O sistema não utiliza banco de dados. A persistência é feita em três arquivos de texto no diretório do usuário, o que atende ao escopo local e simplifica a distribuição.

Arquivo	Formato	Conteúdo
materiais.properties	Chave e valor	Código, nome, densidade, cor e bônus de cada material. Lido na inicialização e carregado na tabela hash do catálogo.
ranking.csv	Valores separados por vírgula	Nome do jogador, pontuação, nível alcançado, linhas eliminadas, número de colapsos e data da partida.
repeticao_<data>.txt	Uma jogada por linha	Sequência de jogadas registradas na pilha de histórico, com o instante, o tipo de comando e a semente do gerador aleatório, permitindo reconstruir a partida integralmente.

2.11. Premissas e restrições

- O jogo será executado em uma única janela, com resolução mínima de 1024 por 768 pixels.
- A entrada se dá exclusivamente por teclado; não há suporte a mouse durante a partida nem a controles externos.
- Toda a simulação de massa é bidimensional e considera blocos de volume unitário; não há simulação de momento fletor, esforço cortante ou deformação.
- O colapso reacomoda blocos por gravidade vertical simples, sem tombamento lateral nem rotação de blocos.
- O sistema utiliza apenas bibliotecas da distribuição padrão do Java, exceto a biblioteca de testes.

2.12. Rastreabilidade entre requisitos e conteúdo da disciplina

A tabela seguinte relaciona cada conceito exigido pela disciplina aos requisitos que o exercitam, evidenciando que nenhum deles foi incluído artificialmente.

Conceito	Requisitos	Manifestação no sistema
Herança	RF03, RF04, RF08	Hierarquias Peca e Material, com atributos e comportamento comuns fatorados nas superclasses abstratas.
Polimorfismo	RF04, RF08, RF13	Chamadas a formas(), aoFixar() e bonusLinha() resolvidas em tempo de execução conforme a subclasse concreta.
Encapsulamento	RNF08	Estado do tabuleiro e do motor acessível apenas por métodos públicos que preservam as invariantes do jogo.
Interfaces	RF15, RNF08	Contratos Desenhavel e Atualizavel implementados por todos os elementos participantes do laço de jogo.
Fila	RF03, RF05	Sequência de elementos futuros e fila de comandos de entrada.
Pilha	RF10, RF23, RF24, RF26	Reserva de elemento, histórico de jogadas, desfazer e reprodução de partida.
Lista	RF02, RF12	Composição dos elementos, linhas completas identificadas e blocos desprendidos no colapso.
Tabela hash	RF04, RF21, RF22	Catálogo de materiais e ranking de pontuações por jogador.
Tratamento de	RNF11, RNF12	Leitura defensiva dos arquivos de dados com recuperação por

Conceito	Requisitos	Manifestação no sistema
exceções		valores padrão.
Recursividade	RF17	Propagação da reacomodação de blocos durante o colapso.

3. Estratégias de codificação

3.1. Plataforma e ferramentas

Item	Definição
Linguagem	Java, versão 17 (LTS), com uso de recursos como registros, expressões switch e classes seladas onde couber.
Interface gráfica	Swing, da distribuição padrão. Descartou-se JavaFX por exigir dependência externa desde o Java 11, o que complicaria a distribuição do trabalho.
Construção	Maven, para padronizar a compilação, a execução dos testes e a geração do pacote executável e da documentação.
Testes e documentação	JUnit 5 para os testes automatizados e Javadoc para a documentação do código, ambos gerados pelo Maven.
Versionamento e ambiente	Git com repositório remoto no GitHub; IntelliJ IDEA Community ou Visual Studio Code, à escolha de cada integrante.

3.2. Arquitetura adotada

O sistema segue o padrão Modelo-Visão-Controle, com uma quarta camada isolada para persistência. A regra que orienta toda a separação é simples e verificável: o pacote de modelo não pode conter nenhuma instrução de importação de classes gráficas. Se essa regra for respeitada, é possível instanciar o motor do jogo, executar milhares de jogadas e conferir o resultado sem abrir uma janela, o que viabiliza os testes automatizados exigidos pelo requisito RNF10.

Arquitetura em camadas do sistema CANTEIRO

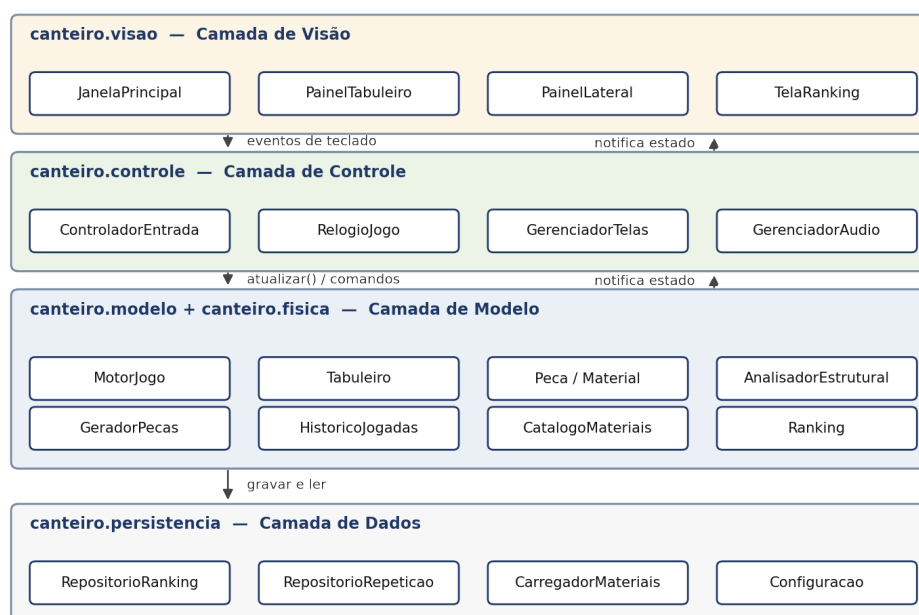


Figura 6 — Arquitetura em camadas e principais fluxos de comunicação.

A comunicação da camada de modelo para a de visão é feita por notificação, e não por chamada direta. O motor mantém uma lista de observadores e emite eventos de mudança de estado; os painéis se inscrevem nessa lista. Assim, o modelo não conhece a existência dos painéis, apenas do contrato de observador — inversão de dependência que preserva a separação de camadas.

3.3. Convenções de código e documentação

- Nomes de classes em notação Pascal, métodos e atributos em notação camelo, constantes em maiúsculas com sublinhado.
- Todo o código, incluindo identificadores e comentários, será escrito em português, com exceção dos termos consagrados da linguagem.
- Atributos sempre privados; acesso externo apenas por métodos que preservem as invariantes da classe.
- Nenhum método com mais de 40 linhas úteis; nenhuma classe com mais de 400 linhas, conforme RNF09.
- Números mágicos proibidos: dimensões, limites e intervalos ficam em constantes nomeadas ou em arquivo de configuração.
- Toda classe pública deve conter, em seu Javadoc, a descrição da responsabilidade, o autor e a versão. Todo método público deve documentar parâmetros, retorno e exceções lançadas.

Exemplo do padrão de documentação a ser seguido:

```
/**
 * Recalcula o centro de massa horizontal da estrutura.
 *
 * <p>Cálculo incremental: usa os acumuladores de massa por coluna,
 * atualizados a cada bloco fixado ou removido (RNF02).</p>
 *
 * @param tabuleiro estrutura avaliada; não pode ser nulo
 * @return posição horizontal do centro de massa, em colunas
 * @throws IllegalStateException se a estrutura não possuir blocos
 * @see #indiceEstabilidade()
 */
public double centroDeMassa(Tabuleiro tabuleiro) { ... }
```

3.4. Uso de herança e polimorfismo

O ponto em que o polimorfismo mais economiza código é o tratamento das rotações. Sem ele, o motor precisaria de uma cadeia de condicionais para descobrir qual matriz de ocupação usar. Com a hierarquia, o motor apenas pede a forma à peça:

```
public abstract class Peca {
    protected Material material;
    protected int rotacao;

    /** Matrizes de ocupação de cada rotação, definidas pela subclasse. */
    protected abstract int[][][] formas();

    public void girarHorario() {
        rotacao = (rotacao + 1) % formas().length;
    }
}
```

```

    public double massaTotal() {
        return blocos.size() * material.massaPorBloco();
    }
}

public class PecaT extends Peca {
    private static final int[][][] FORMAS = { /* quatro rotações */ };

    @Override
    protected int[][][] formas() { return FORMAS; }
}

```

O mesmo princípio se aplica aos materiais. O método `aoFixar` permite que cada material produza um efeito distinto no instante do assentamento, sem que o motor conheça esses efeitos:

```

public abstract class Material {
    protected final double densidade;

    /** Efeito aplicado no instante em que a peça é assentada. */
    public abstract void aoFixar(Tabuleiro tabuleiro, Peca peca);

    public abstract int bonusLinha();
}

public class Madeira extends Material {
    @Override
    public void aoFixar(Tabuleiro tabuleiro, Peca peca) {
        // leve: não altera a vizinhança
    }
    @Override public int bonusLinha() { return 10; }
}

public class Aco extends Material {
    @Override
    public void aoFixar(Tabuleiro tabuleiro, Peca peca) {
        // pesado: consolida os blocos imediatamente abaixo,
        // que passam a resistir ao desprendimento no colapso
        tabuleiro.consolidarAbaixo(peca);
    }
    @Override public int bonusLinha() { return 40; }
}

```

3.5. Algoritmos centrais

3.5.1. Ciclo de atualização

O laço do jogo é conduzido por um temporizador do Swing configurado para aproximadamente 16 milissegundos, o que produz 60 atualizações por segundo. A entrada do teclado não é processada no instante do evento, e sim enfileirada e consumida no início do quadro seguinte, o que evita que a mesma peça sofra alterações concorrentes de estado e mantém o comportamento reprodutível — condição necessária para o recurso de repetição. Cada quadro executa a seguinte sequência:

- Consumir os comandos acumulados na fila de entrada e aplicar movimento ou rotação, sempre precedidos de teste de colisão.
- Verificar se o intervalo de queda do nível se esgotou; em caso negativo, seguir direto ao redesenho.

- Descer a peça uma linha e testar colisão inferior; não havendo colisão, seguir ao redesenho.
- Havendo colisão, fixar a peça no tabuleiro, registrar a jogada na pilha de histórico, eliminar as linhas completas e apurar a pontuação.
- Atualizar os acumuladores de massa, recalcular o centro de massa e o índice de estabilidade.
- Se o desvio ultrapassar o limite do nível, executar o colapso e aplicar a penalidade.
- Retirar o próximo elemento da fila e redesenhar a tela.

A separação entre a etapa de lógica e a etapa de desenho é deliberada e atende ao requisito RNF01: em máquinas mais lentas, é o redesenho que pode ser sacrificado, nunca a atualização do estado, sob pena de o jogo passar a se comportar de forma diferente conforme o equipamento.

3.5.2. Detecção de colisão

A colisão é verificada por tentativa: o motor calcula a posição candidata e consulta o tabuleiro. Como cada peça possui quatro blocos, a verificação percorre quatro posições, sendo de custo constante. A ordem dos testes importa: os limites do tabuleiro são verificados antes do acesso à matriz, evitando índices inválidos.

```
public boolean colide(Peca peca, int linha, int coluna) {
    for (Bloco b : peca.blocosNaPosicao(linha, coluna)) {
        if (b.getColuna() < 0 || b.getColuna() >= COLUNAS) return true;
        if (b.getLinha() >= LINHAS) return true;
        int l = b.getLinha();
        if (l >= 0 && grade[l][b.getColuna()] != null) return true;
    }
    return false;
}
```

3.5.3. Rotação com deslocamento corretivo

Girar uma peça encostada em uma parede resulta, na implementação ingênua, em rotação recusada — comportamento que o jogador percebe como travamento. A estratégia adotada é tentar a rotação e, havendo colisão, testar uma sequência ordenada de pequenos deslocamentos: uma coluna à esquerda, uma à direita, duas à esquerda, duas à direita e uma linha acima. A primeira posição válida é aplicada; se nenhuma for, a rotação é descartada. A sequência de deslocamentos é definida por peça, já que a peça l, por ser longa, exige tentativas mais amplas.

3.5.4. Cálculo incremental do centro de massa

Recalcular a média ponderada percorrendo as duzentas posições do tabuleiro a cada quadro atenderia à correção, mas desperdiçaria processamento. Como o requisito RNF02 exige custo constante, o sistema mantém dois acumuladores: a massa total da estrutura e a soma dos momentos, isto é, a soma dos produtos entre a massa de cada bloco e sua coluna. Ao fixar um bloco, ambos são incrementados; ao remover, decrementados. O centro de massa é a razão entre eles.

```
public void registrarBloco(Bloco b) {
    double m = b.massa();
    massaTotal += m;
    momentoX += m * (b.getColuna() + 0.5);
}

public void removerBloco(Bloco b) {
    double m = b.massa();
    massaTotal -= m;
    momentoX -= m * (b.getColuna() + 0.5);
}
```

```
}  
  
public double centroDeMassa() {  
    if (massaTotal <= 0) throw new IllegalStateException("estrutura vazia");  
    return momentoX / massaTotal;  
}
```

A eliminação de uma linha exige atenção: os blocos eliminados devem ser descontados, e os blocos que descem não alteram o momento horizontal, pois permanecem na mesma coluna. Essa observação simplifica bastante a rotina de eliminação.

3.5.5. Colapso estrutural

Constatado o desvio excessivo, o sistema identifica a linha crítica — a linha imediatamente abaixo do ponto onde a concentração de massa se torna assimétrica — e desprende todos os blocos acima dela. Os blocos desprendidos são então reacomodados coluna a coluna, descendo até encontrar apoio. A reacomodação é implementada de forma recursiva por coluna, o que exercita o conceito de recursividade previsto na rastreabilidade da seção 2.12.

3.6. Tratamento de exceções

A estratégia distingue dois tipos de falha. Falhas de programação, como o cálculo de centro de massa em estrutura vazia, são sinalizadas por exceções não verificadas e devem interromper a execução durante o desenvolvimento, pois indicam defeito. Falhas de ambiente, como ausência ou corrupção dos arquivos de dados, são tratadas e recuperadas: o sistema registra a ocorrência em log e prossegue com valores padrão, conforme exige o requisito RNF11. O jogador nunca deve perder uma partida por causa de um arquivo de ranking malformatado.

3.7. Estratégia de testes

A verificação segue a pirâmide usual: a maior parte do esforço concentra-se em testes de unidade sobre a camada de modelo, que é onde residem as regras passíveis de erro silencioso, e apenas o comportamento de integração das telas é verificado manualmente. A escrita dos testes acompanha a implementação de cada regra, e não é postergada para o fim do desenvolvimento. O detalhamento dos casos previstos consta do plano de verificação apresentado na seção 4.8.

3.8. Controle de versão e trabalho em equipe

O repositório adotará ramificação por funcionalidade: cada integrante trabalha em um ramo próprio e integra ao ramo principal por solicitação de incorporação revisada por outro integrante. A prática reduz conflitos de edição simultânea e produz, no histórico do repositório, o registro individual de contribuição exigido pela avaliação da disciplina.

4. Resultados esperados

4.1. Produto final previsto

Ao término do desenvolvimento, espera-se entregar um aplicativo executável em arquivo único, iniciável por duplo clique em qualquer sistema com Java 17 instalado, contendo o jogo completo, o ranking persistente e a documentação Javadoc gerada. O jogo deve ser jogável por uma pessoa sem instruções prévias além da legenda de teclas exibida na própria tela.

4.2. Interface prevista

A tela principal divide-se em três regiões. Ao centro, o tabuleiro de dez colunas por vinte linhas, com o eixo da base indicado por linha tracejada e o centro de massa marcado por um indicador que se desloca conforme a estrutura cresce. À esquerda, o painel de análise estrutural e o painel de pontuação. À direita, a fila de próximos elementos e o elemento reservado. A Figura 7 apresenta o esboço dessa disposição.

Figura — Esboço da tela principal de jogo

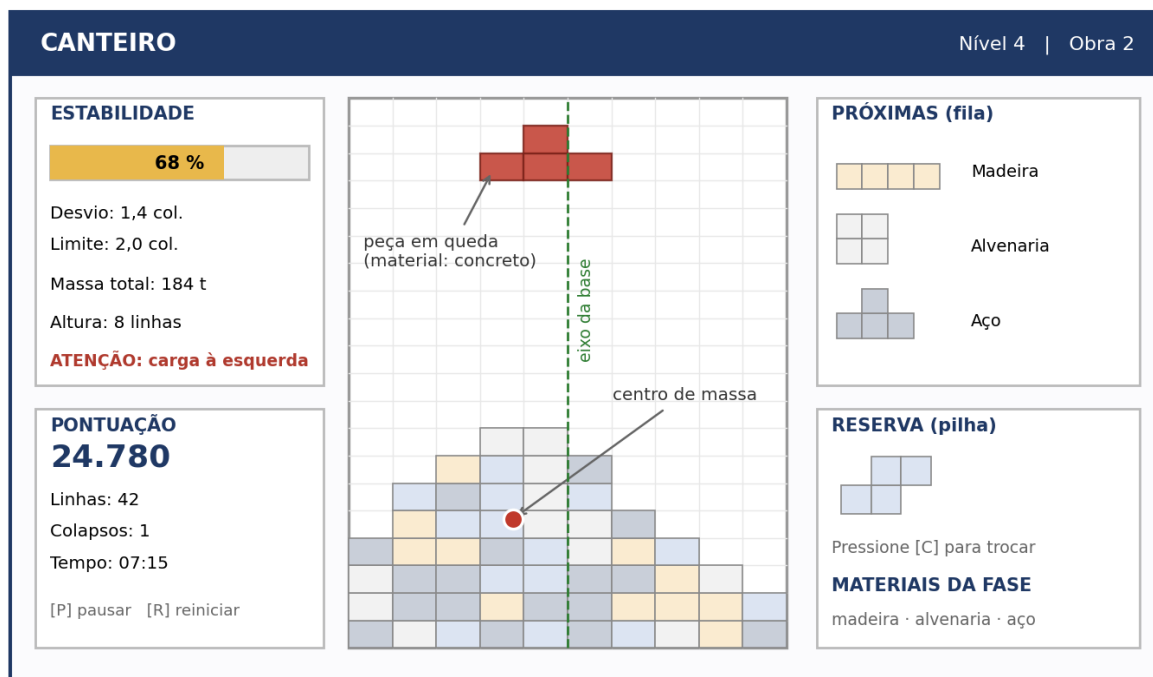


Figura 7 — Esboço da tela principal de jogo.

A escolha de manter o indicador de estabilidade permanentemente visível, e não apenas no momento do alerta, é deliberada: o jogador precisa aprender a associar suas decisões de posicionamento ao deslocamento do indicador, e isso só ocorre se a realimentação for contínua.

4.2.1. Telas previstas

Tela	Conteúdo e comportamento esperado
Menu principal	Título, opções de nova partida, ranking, repetições, configurações e sair. Navegação por teclado e por mouse.
Seleção de dificuldade	Três opções, com apresentação da velocidade inicial, do limite de desvio e dos materiais liberados em cada uma.
Partida	Tela descrita na Figura 7, com atualização contínua e sem oscilação perceptível de taxa de quadros.
Pausa	Sobreposição translúcida sobre o tabuleiro, com o tabuleiro congelado e as opções de retomar, reiniciar e sair.
Colapso	Animação de aproximadamente um segundo mostrando o desprendimento e a acomodação dos blocos, seguida da retomada automática.
Fim de partida	Pontuação final, nível alcançado, linhas eliminadas, número de colapsos, campo para o nome e opção de salvar a repetição.

Tela	Conteúdo e comportamento esperado
Ranking	Dez melhores pontuações com nome, pontos, nível e data, ordenadas de forma decrescente.
Relatório	Gráfico da evolução do índice de estabilidade e distribuição percentual dos materiais utilizados na partida.

4.3. Comportamento esperado da mecânica de estabilidade

A Figura 8 ilustra o resultado esperado do cálculo em duas situações contrastantes. À esquerda, uma estrutura construída de forma simétrica mantém o centro de massa praticamente sobre o eixo da base, com desvio nulo e índice de estabilidade próximo do máximo. À direita, a concentração de blocos de aço e concreto em um dos lados desloca o centro de massa além do limite tolerado e dispara o colapso, ainda que a estrutura ocupe menos posições do tabuleiro.

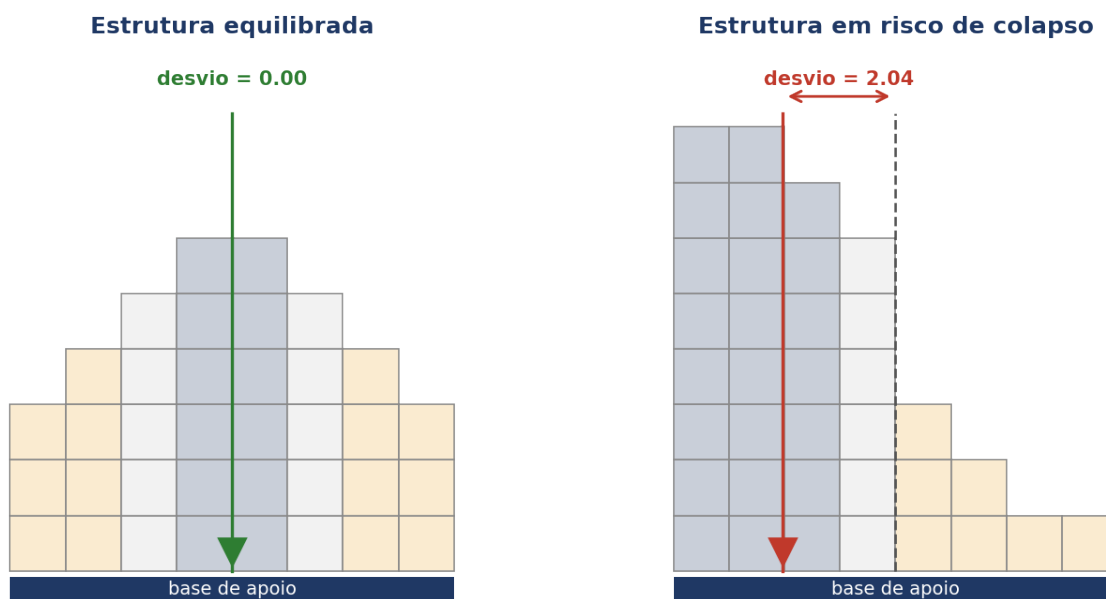


Figura 8 — Comportamento esperado do cálculo de centro de massa em duas configurações.

O contraste entre as duas situações é o resultado pedagógico mais importante do projeto: a segunda estrutura tem menos blocos que a primeira e, ainda assim, é a que falha. É esse comportamento contraintuitivo que se espera que o jogador internalize ao longo das partidas.

4.4. Progressão e balanceamento previstos

O balanceamento inicial parte dos valores da tabela abaixo e será ajustado durante os testes de jogabilidade. A premissa é que a dificuldade cresça por duas vias simultâneas — a velocidade de queda e o aperto do limite de desvio — de modo que o jogador experiente encontre desafio mesmo dominando a mecânica de encaixe.

Nível	Intervalo de queda	Limite de desvio	Materiais liberados	Linhas para avançar
1 a 2	800 ms	3,0 colunas	Madeira, alvenaria	10
3 a 4	650 ms	2,5 colunas	Madeira, alvenaria, concreto	10

Nível	Intervalo de queda	Limite de desvio	Materiais liberados	Linhas para avançar
5 a 6	500 ms	2,0 colunas	Todos	10
7 a 8	380 ms	1,7 coluna	Todos	10
9 a 10	280 ms	1,4 coluna	Todos, com aço mais frequente	10
11 em diante	200 ms	1,2 coluna	Todos, com aço mais frequente	10

4.5. Relatório de fim de partida

Ao encerrar a partida, o sistema apresentará a evolução do índice de estabilidade ao longo dos elementos fixados, marcando os instantes em que houve colapso. O objetivo é dar ao jogador uma leitura do próprio desempenho que vá além da pontuação: uma partida com pontuação alta obtida à custa de dois colapsos revela um estilo de jogo diferente de uma partida com pontuação semelhante e nenhum colapso.

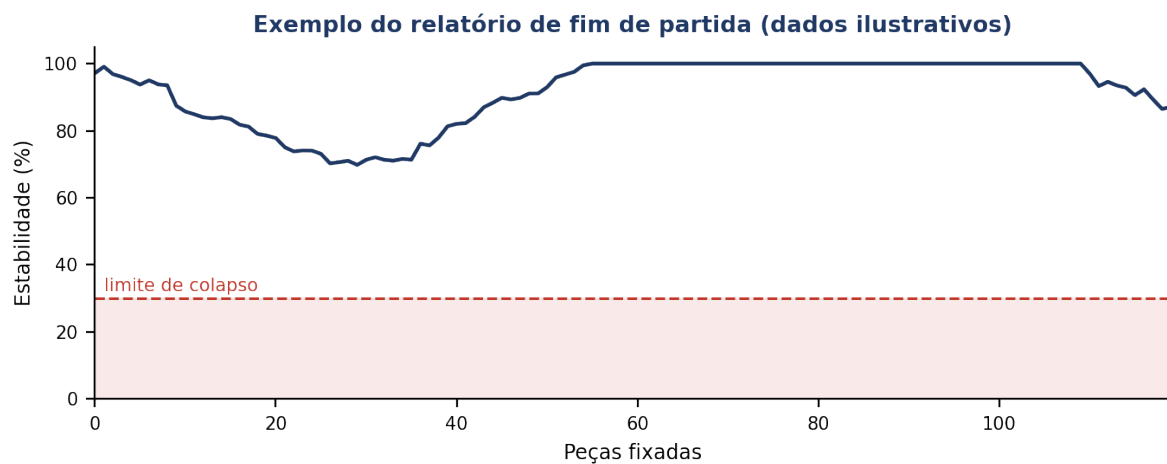


Figura 9 — Formato previsto do gráfico de estabilidade do relatório final. Os dados são ilustrativos.

4.6. Critérios de aceitação

Cada critério abaixo é verificável por observação direta ou por execução de teste, e servirá de base para a validação da entrega final.

Req.	Critério de aceitação	Verificação
RF03	Ao longo de 200 elementos gerados, nenhum formato aparece duas vezes antes que os outros seis tenham aparecido.	Teste automatizado sobre o gerador
RF07	Movimentos laterais junto às paredes são recusados sem que a peça atravesse a borda ou desapareça.	Roteiro manual de jogo
RF08	Toda peça encostada em qualquer das paredes consegue girar, exceto quando as cinco tentativas de deslocamento falharem.	Teste automatizado por forma
RF12	A eliminação de quatro linhas simultâneas resulta em pontuação superior à soma de quatro eliminações isoladas.	Teste automatizado de pontuação
RF14	O centro de massa calculado de forma incremental coincide com o obtido por varredura completa, com tolerância de 0,001.	Teste automatizado comparativo
RF15	O índice exibido acompanha a alteração do tabuleiro no mesmo	Inspeção visual em execução

Req.	Critério de aceitação	Verificação
	quadro em que a peça é fixada.	
RF17	Uma estrutura montada deliberadamente com aço em uma única lateral entra em colapso em no máximo doze peças.	Roteiro manual reproduzível
RF21	Pontuações registradas permanecem após o encerramento e a reabertura do aplicativo.	Roteiro manual
RNF01	A taxa de quadros permanece acima de 55 quadros por segundo durante dez minutos de partida contínua.	Medição por contador interno
RNF07	A geração da documentação Javadoc conclui sem advertências de elementos não documentados.	Execução do processo de construção

4.7. Métricas de desempenho esperadas

As metas abaixo derivam dos requisitos não funcionais e serão medidas por instrumentação do próprio jogo, que registrará em log os tempos de cada etapa do ciclo de atualização durante as sessões de teste.

Métrica	Meta	Forma de medição
Taxa de atualização	≥ 55 qps	Contador de quadros por segundo exibido em modo de depuração, amostrado a cada segundo.
Tempo do ciclo de atualização	≤ 4 ms	Cronometragem do trecho de lógica, separada do tempo de desenho.
Latência de resposta ao teclado	≤ 50 ms	Registro do instante do evento e do instante do redesenho correspondente.
Recálculo do centro de massa	Tempo constante	Comparação do tempo médio com a pilha em 2, 10 e 18 linhas de altura.
Tempo de inicialização	≤ 2 s	Do acionamento do executável à exibição do menu principal.
Memória em uso	≤ 200 MB	Monitoramento da máquina virtual após dez minutos de partida contínua.

4.8. Plano de verificação e testes

A tabela seguinte relaciona os casos de teste previstos para cada regra crítica do sistema. Os casos marcados como automatizados serão implementados em JUnit e executados a cada integração ao ramo principal.

Alvo	Casos previstos	Tipo
Colisão	Peça encostada em cada uma das quatro bordas; peça sobre bloco já fixado; peça em espaço livre; peça parcialmente acima do topo do tabuleiro.	Automatizado
Rotação	As quatro rotações de cada uma das sete formas; rotação junto às paredes esquerda e direita; rotação da peça I em espaço mínimo; rotação recusada por falta de espaço.	Automatizado
Geração de peças	Distribuição dos formatos em 200 gerações; nenhuma repetição antes de esgotada a sacola; sorteio de material restrito aos liberados na fase.	Automatizado
Eliminação de linhas	Uma, duas, três e quatro linhas simultâneas; linha completa não adjacente ao topo da pilha; nenhuma linha completa; deslocamento correto das linhas superiores.	Automatizado

Alvo	Casos previstos	Tipo
Centro de massa	Estrutura simétrica com material único; estrutura simétrica em geometria mas assimétrica em massa; estrutura de coluna única; confronto entre o cálculo incremental e a varredura completa após 500 operações aleatórias.	Automatizado
Colapso	Desvio imediatamente abaixo do limite; desvio imediatamente acima; colapso com a pilha na altura crítica; reacomodação correta dos blocos desprendidos.	Automatizado
Pontuação	Multiplicadores por número de linhas, por material e por nível; penalidade de colapso; avanço de nível a cada dez linhas.	Automatizado
Persistência	Arquivo ausente; arquivo vazio; linha malformada; caractere inválido; ranking com nome repetido; escrita concorrente.	Automatizado
Repetição de partida	Reexecução de uma partida gravada deve reproduzir exatamente a mesma pontuação e o mesmo estado final do tabuleiro.	Automatizado
Interface	Navegação entre todas as telas; pausa e retomada; redimensionamento da janela; legibilidade das cores em modo de alto contraste.	Manual

O caso de confronto entre o cálculo incremental e a varredura completa do tabuleiro merece destaque. É a forma mais econômica de detectar erros de sincronização dos acumuladores de massa, que constituem a falha mais provável dessa otimização e que, se passassem despercebidos, produziram colapsos aparentemente injustos — o tipo de defeito que o jogador percebe como jogo quebrado sem conseguir descrever a causa.

4.9. Validação com jogadores

A partir da semana 8, quando a mecânica de estabilidade estiver funcional, serão conduzidas sessões de observação com pelo menos seis pessoas externas ao grupo, sem instrução prévia além da legenda de teclas. Cada sessão terá duração aproximada de quinze minutos e registrará: o tempo até o participante compreender a relação entre a distribuição de material e o indicador de estabilidade; o número de colapsos sofridos por partida; e os momentos em que o participante demonstrou não compreender por que a estrutura desabou.

O terceiro registro é o mais importante para o ajuste do sistema. Um colapso que o jogador não consegue explicar indica falha de comunicação da interface, não de compreensão do jogador, e deverá ser tratado com realimentação visual adicional — por exemplo, destacando a região da estrutura que concentra a massa antes que o limite seja atingido.

4.10. Entregáveis previstos

Entregável	Descrição
Código-fonte	Arquivo compactado com o projeto Maven completo, incluindo os testes automatizados e os arquivos de dados de exemplo.
Executável	Arquivo JAR único, com dependências embutidas, iniciável por duplo clique.
Documentação Javadoc	Arquivo compactado com o conjunto de páginas geradas a partir dos comentários do código.
Relatório de plataforma e desvios	Documento com o ambiente de codificação utilizado, as modificações feitas em relação a estes requisitos e a justificativa de cada uma.
Esta documentação	Documento de especificação em formato PDF.

4.11. Cronograma previsto

Período	Atividade	Produto
Semanas 1 e 2	Levantamento de requisitos, modelagem de classes e definição da arquitetura.	Este documento
Semanas 3 e 4	Implementação do modelo: peças, materiais, tabuleiro e colisão, com testes.	Núcleo funcional em modo texto
Semanas 5 e 6	Motor do jogo, rotação com deslocamento corretivo, eliminação de linhas e pontuação.	Jogo jogável sem estabilidade
Semanas 7 e 8	Análise estrutural: acumuladores, centro de massa, índice e colapso.	Mecânica diferencial completa
Semanas 9 e 10	Interface gráfica, telas, animações e realimentação visual.	Versão gráfica integrada
Semana 11	Persistência, ranking, repetição e relatório final.	Requisitos desejáveis
Semana 12	Testes de jogabilidade, balanceamento, Javadoc e empacotamento.	Entrega final

4.12. Riscos identificados

Risco	Probabilidade e impacto	Ação prevista
A mecânica de colapso tornar o jogo frustrante ou injusto	Média / Alto	Realizar testes com pessoas de fora do grupo já na semana 8 e ajustar o limite de desvio antes de prosseguir.
Oscilação de taxa de quadros no Swing em máquinas mais lentas	Média / Médio	Separar a lógica de atualização do desenho e permitir redução da resolução do tabuleiro.
Subestimação do esforço da rotação com deslocamento corretivo	Alta / Médio	Implementar primeiro a rotação simples e adicionar os deslocamentos como incremento, mantendo o jogo funcional em qualquer momento.
Desequilíbrio na divisão do trabalho entre integrantes	Média / Alto	Revisão semanal do quadro de tarefas e registro individual de contribuições no repositório.
Ausência de integrante em apresentação de etapa	Baixa / Alto	Confirmação das datas em calendário compartilhado e ensaio da apresentação com dois dias de antecedência.

4.13. Limitações reconhecidas

O modelo físico adotado é uma simplificação deliberada. Não há simulação de esforços internos, de resistência dos materiais, de deformação nem de tombamento com rotação. O critério de colapso considera apenas o deslocamento horizontal do centro de massa, o que é suficiente para produzir o comportamento de jogo desejado, mas não constitui análise estrutural no sentido da engenharia. Essa limitação será explicitada na própria tela do jogo, para evitar que a ferramenta seja interpretada como instrumento de cálculo.

5. Conclusão

Este documento apresentou a especificação do CANTEIRO, um jogo de encaixe de blocos em que a estrutura construída pelo jogador precisa permanecer equilibrada sobre sua base de apoio. A proposta partiu de um gênero consolidado e amplamente implementado, e buscou nele um ponto de inserção original: a atribuição de massa às peças e a consequente sujeição da pilha a uma restrição de equilíbrio.

O levantamento de requisitos produziu vinte e oito requisitos funcionais e doze não funcionais, ancorados em quinze regras de negócio, e a matriz de rastreabilidade da seção 2.12 evidencia que cada conceito exigido pela disciplina encontra no sistema um uso legítimo. A fila existe porque há uma sequência de peças a consumir em ordem; a pilha existe porque a reserva e o desfazer operam sobre o último elemento inserido; a tabela hash existe porque materiais são recuperados por código milhares de vezes ao longo de uma partida. Nenhuma dessas estruturas foi introduzida para satisfazer o enunciado.

As duas hierarquias de classes, separando forma e material, ilustram o ganho concreto que herança e polimorfismo oferecem: a alternativa sem elas exigiria vinte e oito classes ou uma extensa cadeia de condicionais no motor do jogo, e qualquer material novo obrigaria a alterar código existente. Com a modelagem proposta, acrescentar um quinto material significa escrever uma classe e registrá-la no catálogo.

As estratégias de codificação priorizaram a testabilidade. A decisão de proibir dependências gráficas na camada de modelo, embora custe algum esforço de disciplina durante o desenvolvimento, é o que torna possível verificar automaticamente as regras mais delicadas do sistema — em especial a consistência do cálculo incremental do centro de massa, que é simultaneamente o diferencial do projeto e seu ponto mais provável de falha.

Os riscos de maior impacto foram identificados e associados a ações preventivas concretas, com destaque para a validação antecipada da mecânica de colapso junto a pessoas externas ao grupo. Trata-se do risco central da proposta: uma mecânica original é, por definição, uma mecânica não testada, e sua aceitação pelo jogador não pode ser presumida.

Espera-se, ao final do desenvolvimento, entregar um sistema que seja ao mesmo tempo um jogo efetivamente divertido e uma demonstração honesta dos conceitos estudados na disciplina, em que a escolha de cada estrutura de dados e de cada relação de herança possa ser justificada por uma necessidade real do problema, e não pela exigência de um enunciado.